



Rifugio per adozione animali

<http://localhost/tmoretti/>

Relazione di progetto

Università degli Studi di Padova
L.T. in Informatica
A.A. 2025/26

2111001 (Referente) Thomas Morettin (thomas.morettin@studenti.unipd.it)
2111935 Felician Mario Necsulescu (felicianmario.necsulescu@studenti.unipd.it)
2111947 Niccolò Feltrin (niccolo.feltrin@studenti.unipd.it)

Data 02.02.2026
Corso Tecnologie Web
Cod. SCP4065581

Credenziali di accesso al sito

Ruolo	Username	Password
Amministratore	admin	admin

Indice

1	Scopo del sito	2
2	Analisi dei requisiti	2
2.1	Analisi del target	2
2.2	Tipi di utente	2
2.3	Funzionalità	3
3	Progettazione	3
3.1	Struttura del sito	3
3.1.1	Sistema di template centralizzato	3
3.1.2	Scheletro comune	4
3.1.3	Organizzazione delle pagine	4
4	Web Development	5
4.1	HTML5	5
4.2	CSS3	5
4.3	JavaScript	6
4.4	PHP	6
4.4.1	Pattern MVC	6
4.4.2	utils.php	7
4.4.3	Composer (vendor)	7
4.4.4	db-access.php	7
4.4.5	template.php, header.php e breadcrumb.php	7
4.4.6	.htaccess	8
4.5	Base di dati	8
4.6	Sicurezza	9
5	Accessibilità e usabilità	9
5.1	Elementi fondamentali	9
5.2	Orientamento e navigazione	10
5.3	WAI-ARIA	10
5.4	Colori e Brand Identity	11
5.5	Convenzioni interne del sito	12
6	Test e validazione	12
7	SEO e prestazioni	14
7.1	faq-schema.js	15
7.2	org-schema.js	15
8	Organizzazione del lavoro	15
9	Conclusioni e difficoltà incontrate	16

1 Scopo del sito

Amici per Sempre è una piattaforma web progettata per il rifugio di animali domestici situato a Padova. Il sito rappresenta la vetrina digitale della struttura e offre servizi completi sia per chi cerca un animale da adottare sia per chi ha necessità di affidare il proprio animale al rifugio. L'obiettivo principale è duplice:

- **Lato utenti:** permettere la consultazione degli animali ospitati, prenotare visite conoscitive per l'adozione e richiedere l'inserimento del proprio animale nel rifugio;
- **Lato rifugio:** gestire in modo efficiente le richieste di adozione, il calendario degli appuntamenti per visite e per l'accoglienza di nuovi animali.

Il portale si propone di creare un ponte bidirezionale: da un lato gli animali in cerca di famiglia incontrano i potenziali adottanti, dall'altro le persone che per vari motivi non possono più tenere il proprio animale trovano una soluzione responsabile attraverso l'affidamento al rifugio.

2 Analisi dei requisiti

2.1 Analisi del target

Il sito è progettato per un'utenza eterogenea che include:

- **Potenziali adottanti:** persone di tutte le età in cerca di un animale da compagnia;
- **Proprietari in difficoltà:** cittadini che per motivi personali, economici o di salute necessitano di affidare il proprio animale a una struttura qualificata;
- **Visitatori curiosi:** utenti che desiderano conoscere la realtà del rifugio;
- **Amministratori:** personale del rifugio che gestisce richieste e appuntamenti.

Per garantire accessibilità universale, l'interfaccia adotta un linguaggio semplice, una gerarchia visiva chiara e una navigazione intuitiva, riducendo il carico cognitivo anche per utenti con competenze informatiche limitate o in situazioni di stress emotivo (come chi deve separarsi dal proprio animale).

2.2 Tipi di utente

Il sistema prevede due categorie principali di utenti con permessi differenziati:

Utente non autenticato:

- Visualizzazione degli animali disponibili per l'adozione con sistema di filtri avanzato;
- Accesso alle informazioni sul rifugio e sulla missione;
- Compilazione del form "Porta in Adozione" per richiedere un appuntamento e cedere il proprio animale;
- Possibilità di consultare le schede dettagliate di ogni animale e prenotare una visita compilando un form.

Utente autenticato (amministratore):

- Accesso all'area amministrativa con dashboard riepilogativa;
- Gestione completa del calendario appuntamenti (visite per adozione + accoglienza animali esterni);
- Visualizzazione e gestione dei ticket di richiesta adozione;
- Gestione delle richieste di inserimento animali esterni nel rifugio;

2.3 Funzionalità

Il portale offre un insieme articolato di funzionalità pensate per rispondere ai bisogni dei diversi tipi di utenti:

- **Catalogo animali dinamico.** La sezione principale del sito consente la visualizzazione degli animali disponibili per l'adozione tramite un catalogo dinamico. Gli utenti possono filtrare i risultati per specie, sesso, età, taglia e carattere, oppure cercare un animale per nome tramite una barra di ricerca testuale. Un contatore aggiorna in tempo reale il numero di risultati visualizzati.
- **Schede dettagliate degli animali.** Ogni animale dispone di una scheda dedicata contenente informazioni strutturate, una breve storia descrittiva e un elenco di caratteristiche comportamentali. I dati principali includono specie, età formattata automaticamente, sesso, peso, colore e razza, con particolare attenzione all'accessibilità tramite l'uso dell'attributo `lang` per razze con denominazione in lingua straniera.
- **Form “Porta in Adozione”.** Il sistema permette ai proprietari di animali di richiedere l'inserimento nel rifugio tramite un form guidato in tre fasi. Vengono raccolti i dati del proprietario e le informazioni principali sull'animale, con validazione sia lato client che lato server. Al termine della procedura viene inviata un'email di conferma e creato un appuntamento per la visita di accoglienza.
- **Sistema di prenotazione visite.** Gli utenti autenticati possono prenotare una visita conoscitiva direttamente dalla scheda di un animale. Il form raccoglie i dati di contatto dell'utente e associa automaticamente la richiesta all'animale selezionato. A seguito dell'invio, il sistema genera un ticket di adozione e invia una conferma via email.
- **Dashboard amministrativa.** L'area amministrativa presenta una dashboard riepilogativa con statistiche aggiornate in tempo reale. Vengono mostrati il numero di appuntamenti, ticket di adozione pendenti e richieste di inserimento, oltre a collegamenti rapidi alle principali sezioni di gestione.
- **Calendario appuntamenti.** Il calendario consente la gestione degli appuntamenti tramite una vista mensile e settimanale accessibile. Gli appuntamenti sono distinti visivamente in base alla tipologia e possono essere modificati tramite finestre di dialogo completamente navigabili da tastiera.
- **Gestione ticket adozione.** Questa sezione permette di visualizzare e gestire le richieste di adozione organizzate per animale. L'amministratore può consultare i dati completi del richiedente e approvare o rifiutare le richieste tramite dialog di conferma, facilitando una gestione ordinata e sicura.
- **Richieste inserimento rifugio.** Le richieste provenienti dal form “Porta in Adozione” sono raccolte in una sezione dedicata all'amministratore. Vengono mostrati i dati del proprietario e dell'animale esterno, con la possibilità di accettare la richiesta o rifiutarla previa conferma.

3 Progettazione

3.1 Struttura del sito

L'organizzazione del portale è stata studiata per assicurare un'esperienza di navigazione intuitiva e un facile orientamento tra i contenuti. La piattaforma è articolata in macro-aree funzionali con livelli di accesso distinti, strutturate secondo una gerarchia informativa ben definita.

3.1.1 Sistema di template centralizzato

Il sito adotta un sistema di generazione dinamica delle pagine basato su file di template HTML e script PHP dedicati. Il file `template.html` costituisce lo scheletro base di ogni pagina e include placeholder che vengono sostituiti dinamicamente dal codice PHP con i contenuti specifici di ciascuna pagina. La funzione `buildPage()` nel file `utils.php` coordina l'assemblaggio finale della pagina: riceve come parametri il nome del file HTML contenente il main e un array associativo di dati per la sostituzione dei placeholder. Questa funzione si occupa di caricare il template base tramite `buildTemplate()`, inserire il contenuto principale, popolare tutti i placeholder con i valori forniti e rimuovere eventuali placeholder non sostituiti. Il file `template.php` gestisce la composizione degli elementi comuni del layout. La funzione

`buildTemplate()` carica il template base e lo popola con header (tramite `populatedNavbar()`), breadcrumb (tramite `populatedBread()`), footer e pulsante account (tramite `getAccountButton()`). Questo approccio garantisce che tutti gli elementi strutturali siano generati in modo coerente su ogni pagina del sito.

3.1.2 Scheletro comune

Tutte le pagine del sito condividono una struttura HTML5 semantica uniforme definita nel file `template.html`. Gli elementi costitutivi principali sono:

- **Head standardizzato.** La sezione `<head>` contiene metadati completi per SEO e accessibilità. Sono presenti meta tag per charset UTF-8, viewport responsive, description e keywords dinamiche. Il meta tag `format-detection` è impostato su `telephone=no` per prevenire l'attivazione automatica di link telefonici su numeri non destinati a questo scopo, evitando disorientamento dell'utente. La sezione include inoltre meta tag Open Graph per l'ottimizzazione della condivisione sui social media, favicon in multiple risoluzioni e formati, link ai fogli di stile con relative versioni mobile e print.
- **Header (`header.html` e `header.php`).** Il componente header costituisce il punto centrale della navigazione e viene generato dinamicamente attraverso la funzione `populatedNavbar()` in `header.php`. La struttura include:
 - Il logo del rifugio, gestito dalla funzione `replaceLogo()`, che genera un `<h1>` semplice nella homepage e un `<h1>` cliccabile nelle altre pagine. Nella homepage il titolo non è linkato per evitare ridondanza, mentre nelle altre pagine diventa un link verso la home. Questa scelta mantiene la convenzione di un unico `<h1>` per pagina riservato al contenuto semanticamente più importante.
 - Il menu di navigazione principale, generato dalla funzione `getNavbarLinks()`, che crea un array di link basandosi sulla pagina corrente rilevata tramite `basename($_SERVER["PHP_SELF"])`. I link disponibili sono Home, Adotta e Porta in adozione.
 - L'area pulsanti di utilità, che include il pulsante per rimozione/ripristino dello sfondo decorativo, il pulsante per cambio tema chiaro/scuro e i pulsanti account. Per questi ultimi pulsanti viene verificato lo stato di autenticazione tramite `is_logged_in()` e restituisce HTML differente: per utenti non autenticati mostra un singolo pulsante Login, mentre per amministratori autenticati mostra due pulsanti (Area amministrativa e Logout).
- **Breadcrumb (`breadcrumb.html` e `breadcrumb.php`).** Il sistema di breadcrumb fornisce orientamento all'utente attraverso la generazione dinamica del percorso di navigazione. Le pagine di errore (401, 404, 500) non presentano breadcrumb come scelta progettuale intenzionale, in quanto non fanno parte del percorso di navigazione standard del sito.
- **Toast di notifica.** Il template include un elemento di output con attributi ARIA appropriati per annunciare messaggi di successo o errore all'utente.
- **Main.** Il contenuto principale specifico di ogni pagina viene inserito dinamicamente tramite il sistema di placeholder. Questo elemento è marcato semanticamente con tag `<main>` e rappresenta il punto di arrivo dei link di salto per la navigazione accessibile.
- **Footer (`footer.html`).** La struttura del footer è statica e include diverse sezioni informative organizzate semanticamente. La sezione "Link utili" contiene collegamenti rapidi a FAQs e Contatti tramite ancore alla homepage. Il copyright con anno dinamico aggiornato da JavaScript chiude la sezione.
- **Pulsante back-to-top.** Un pulsante flottante permette di tornare rapidamente in cima alla pagina. L'elemento viene nascosto automaticamente se JavaScript è disabilitato per evitare di mostrare funzionalità non operative, garantendo progressive enhancement.

3.1.3 Organizzazione delle pagine

La piattaforma è strutturata secondo un modello gerarchico definito nell'array `$nomiPag` del file `breadcrumb.php`, che costituisce la mappa completa della navigazione.

Livello radice: La homepage (`index.php`) rappresenta l'unico elemento senza padre, punto di partenza per tutta la navigazione.

Secondo livello: Direttamente accessibili dalla home sono le pagine Adotta (`adotta.php`), Porta in adozione (`porta-in-adozione.php`), Login (`login.php`) e Amministrazione (`amministrazione.php`).

Terzo livello: La pagina Adotta ha come figlia la Scheda Animale (`scheda_animale.php`). L'area amministrativa si dirama in tre sezioni: Calendario (`calendario.php`), Gestione ticket (`gestione-ticket.php`) e Richieste inserimento rifugio (`richieste-inserimento-rifugio.php`).

Pagine di errore: Le pagine 401, 404 e 500 hanno padre impostato a `null`, escludendole dal sistema di breadcrumb come scelta progettuale per evitare di includerle in un percorso di navigazione standard.

4 Web Development

All'interno di questa sezione si vogliono descrivere le tecnologie, gli standard e la struttura adottati all'interno del progetto.

4.1 HTML5

Le pagine del sito web sono state create facendo uso di HTML5 (HyperText Markup Language 5), ovvero lo standard attuale per la progettazione di siti web. In particolare sono state adottate le seguenti best practices di programmazione:

- SoC (Separation of Concerns), in quanto si è mantenuta la separazione tra la struttura (HTML5), lo stile (CSS) e la logica di funzionamento (JavaScript/PHP), rendendo il prodotto scalabile e manutenibile;
- metatag, ovvero snippet per definire metadati utili al motore di ricerca e ai browser, incrementando la SEO (Search Engine Optimization) e rendendo il sito accessibile;
 - è stato fatto uso, oltre a quelli comunemente utilizzati, del metatag `format-detection` per disabilitare l'abilitazione di un link a numero di telefono, da parte del browser, su un numero che non lo sarebbe (pena disorientamento dell'utente).
- separazione degli elementi, in quanto si è deciso di adottare un approccio scalabile, rendendo elementi come i dialog, l'header, il footer e la breadcrumb unici all'interno di tutto il sito, potendoli riutilizzare in più pagine differenti mantenendone una gestione, sia semantica che di comportamento, centralizzata;
- validazione delle pagine al sito W3C Validator per mantenere una semantica corretta e garantire l'accessibilità.

Con questo linguaggio di markup è stato possibile strutturare il corpo dell'e-mail automatica che viene inviata dal server all'utente che compila una richiesta tramite form all'interno del sito. Allo stato attuale i server di posta non supportano una struttura moderna, ma bensì obsoleta, richiedendo l'uso di tabelle per la strutturazione del contenuto o, peggio, di attributi non coerenti con gli standard correnti. Il tutto, però, è stato gestito con la massima attenzione all'accessibilità e alla struttura, rendendo comunque l'e-mail fruibile e leggibile da chiunque.

4.2 CSS3

Per la presentazione del sito web è stato fatto uso del linguaggio CSS3 (Cascading Style Sheets 3), con la costruzione di layout sia flexbox che grid per mantenere una struttura scalabile e pulita all'interno delle pagine. In particolare, il foglio di stile principale `style.css` viene utilizzato per definire un layout generale desktop, permettendo poi al sito di degradare in modo fluido quando viene visualizzato su schermo inferiore ai 768px di larghezza grazie alle ridefinizioni nel foglio `style-smartphone.css`. Il linguaggio è stato utilizzato, inoltre, per gestire la stampa delle pagine e garantire che vengano impaginate correttamente.

A livello presentazione è stata utilizzata l'unità fissa px solamente per ombre, bordi di elementi e larghezza massima della pagina (1200px); per i restanti elementi (l'estrema maggioranza) solamente unità dinamiche come rem, em, %, e vh.

Con questo linguaggio è stato possibile definire la presentazione del corpo dell'e-mail automatica precedentemente citata. Gli standard in uso per questa tecnologia, come già detto risultano obsoleti e richiedono l'utilizzo di stili inline per poter essere applicati correttamente e adattarsi ai principali servizi di posta. Nonostante ciò è stato possibile mantenere la separazione tra la struttura e la presentazione (`style-email.css`).

4.3 JavaScript

Per la gestione del comportamento delle pagine si è fatto uso di JavaScript. In particolare si è optato per includere all'interno di ciascuna pagina web un file `global.js`, il quale contiene le seguenti funzioni più importanti:

- gestione della funzionalità di rimozione/reinserimento dello sfondo delle pagine (gestita tramite variabile d'ambiente);
- gestione della funzionalità tema chiaro/scuro (gestita tramite variabile d'ambiente);
- gestione della comparsa e definizione grafica dell'elemento toast, utilizzato per l'annuncio all'utente di operazioni andate o meno a buon fine;
- funzione di "caricamento" degli elementi grafici fondamentali, in quanto se il JavaScript dovesse essere disabilitato mostrerebbe elementi all'utente senza un funzionamento (es. con JS disabilitato è impossibile ritornare in cima alla pagina, il pulsante viene nascosto);
- funzione di cambio anno di copyright dinamico.

Successivamente, ciascuna pagina possiede un file JavaScript proprio per la dichiarazione di event listeners, funzioni di controllo degli input e della gestione degli errori. Questa scelta di separazione tra un file globale e quelli specifici è stata adottata per rendere le pagine meno pesanti a livello di memoria, dichiarando elementi solo in caso di necessità. Ulteriori file aggiuntivi possono essere caricati tramite placeholder `{{extra-js}}`, come per esempio il codice che gestisce il funzionamento del pannello di filtri o di script `org` per il potenziamento della SEO.

4.4 PHP

A livello server-side è stato fatto uso di linguaggio PHP (Hypertext PreProcessor). Gli script rappresentano il motore di tutte le pagine del sito, in quanto la creazione della pagina è centralizzata all'interno del file `utils.php`, il quale permette di generare, a partire da un template base, le pagine a struttura standardizzata. Successivamente, le interazioni con la base di dati e la conseguente creazione degli elementi ad essa dipendenti vengono gestite da file PHP dedicati per ciascuna pagina.

4.4.1 Pattern MVC

All'interno dell'ambito programmazione il Model View Controller è il design pattern più rinomato ed efficiente in quanto di gestione ed organizzazione dei compiti per ciascuno script interno al progetto. In particolare si riconoscono le seguenti categorie:

- Model: interfaccia che permette l'interazione con la base di dati; all'interno del progetto afferiscono a questa categoria tutti i file PHP che, dopo aver instaurato la connessione con il DB, hanno il compito di estrapolare i dati tramite query parametrizzate, sanitizzare l'output e restituire allo script chiamante i risultati;
- View: insieme dei file HTML che presentano lo scheletro per la strutturazione dei dati da visualizzare;
- Controller: insieme di script PHP adibiti alla manipolazione dei risultati restituiti dal Model; questi script hanno il compito di generare markup strutturale coerente con i dati ricevuti ed arricchire così lo scheletro HTML con dati strutturati e organizzati; oltre ai file per la creazione di elementi dinamici nella pagina HTML, afferiscono a questa categoria anche i file utilizzati per l'invio dei dati alla base di dati, con controllo dell'input a livello server-side (massimizzando il controllo già previsto lato client tramite JS).

4.4.2 utils.php

Progettato per essere il file "cuore" del motore che permette la costruzione delle pagine esso presenta le seguenti funzionalità principali:

- dichiara univocamente all'interno di tutto il progetto la root del sito web, permettendone la sostituzione al placeholder `{{root}}` o potendo essere riutilizzata all'interno di altri script PHP;
- funzione per il controllo di login da parte dell'utente tramite parametro di sessione e corrispondenza della chiave criptata per la password;
- funzione per la ricezione del messaggio di sessione (success/error) per il popolamento dell'elemento toast; il testo viene caricato all'interno della variabile `data-page` del tag `body`, utilizzabile anche per generici valori di sessione;
- funzione per la costruzione della pagina a partire da un template strutturato all'interno del file `template.html`; nel template vengono caricati: header, breadcrumb e footer (definiti in file `.html` appositi); all'interno di questa funzione si evidenziano i seguenti comportamenti:
 - nel caso in cui sia presente il placeholder `{{extra-js}}` vengono creati i tag di `<script>` appositi per incorporare più file JS contemporaneamente;
 - sostituzione di tutti i placeholder della pagina con eliminazione finale di quelli non sostituiti;
- funzione di parsing del file `request-mail.html`, contenente la struttura dell'email automatica, per "iniettare" i parametri di stile all'interno dei tag HTML tramite libreria Emogrifier; come spiegato in precedenza risulta essere il modo più efficiente per mantenere la compatibilità tra i diversi server di posta;
- funzione per l'invio automatico dell'email alla compilazione dei form di adozione o porta in adozione tramite libreria PHPMailer; l'invio avviene mediante protocollo SMTP, server Gmail (dominio di posta dell'email), con password per applicazione e completamento del destinatario.

4.4.3 Composer (vendor)

È stato fatto uso di Composer come programma applicativo per l'installazione delle librerie necessarie all'interno del progetto di Emogrifier (CSS inline parser) e PHPMailer (composizione automatica dell'email). Questa scelta permette la scalabilità del progetto, in quanto risulta semplice l'integrazione di nuove librerie senza l'obbligo di scaricarle manualmente, e la delegazione del compito di gestione delle dipendenze tra, per l'appunto, librerie.

4.4.4 db-access.php

File PHP per la gestione dell'apertura e chiusura della connessione alla base di dati. Lo script implementa le funzioni di apertura, chiusura e restituzione della connessione, insieme alla funzione di esecuzione della query che ha il compito di gestirne i parametri in ingresso, preparare lo statement e renderlo innocuo a SQL Injection e restituire uno/più risultati in base alla tipologia (SELECT, UPDATE, DELETE) della query.

4.4.5 template.php, header.php e breadcrumb.php

Come descritto precedentemente ciascun elemento della pagina viene gestito distintamente. Il file `template.php` permette di creare lo scheletro contenente tag `<head>` popolato e `<body>` da popolare.

Il file `header.php` presenta le funzioni per popolare i link di navigazione, con loro marcatura come link o meno, per inserire il link alla home nel logo e generare i pulsanti di amministrazione nel caso l'utente si sia autenticato.

Il file `breadcrumb.php` contiene al suo interno la definizioni dell'albero dei percorsi e come devono essere formattati all'interno dell'elemento breadcrumb, fondamentale per evitare il disorientamento dell'utente.

4.4.6 .htaccess

Il seguente file permette le operazioni di rooting del sito web. Nello specifico permette:

- visualizzazione delle pagine di errore personalizzate al posto della schermata predefinita del server per gli errori 500 e 404;
- pulizia dell'URL di pagina con rimozione dell'estensione `.php`;
- definizione dell'albero delle pagine, come descritto dall'elemento breadcrumb, senza possibilità di visualizzare i percorsi interni al repository di progetto.

4.5 Base di dati

Per il corretto funzionamento del sito web è stata implementata una base di dati in linguaggio MySQL all'interno del RDBMS Maria DB, la quale presenta al suo interno tutte le entità fondamentali per gestire le richieste degli utenti per l'adozione degli animali e la possibilità di portarli in adozione al rifugio; il tutto può essere centralizzato per l'amministratore tramite gestione delle richieste, una volta programmate, in calendario.

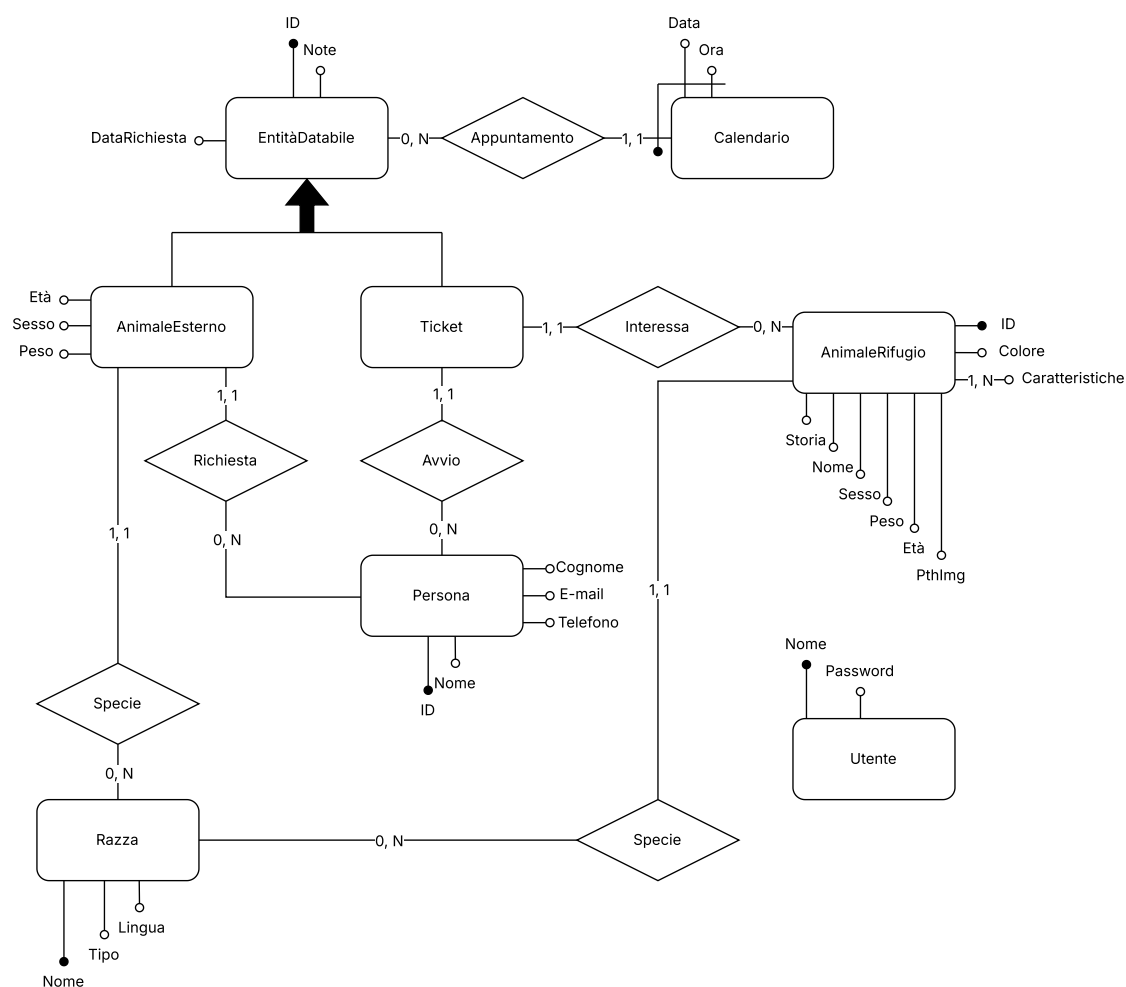


Figura 1: Diagramma ER della base di dati

Per questioni di spazio, ma volendo comunque approfondire il diagramma soprariportato, si evidenziano le seguenti scelte progettuali:

- l'entità **EntitàDatabile** viene utilizzata per accorpare gli attributi comuni di **AnimaleEsterno** e **Ticket**, ovvero la richiesta di adozione; in particolare l'entità serve per permettere di calendarizzare le tuple, delle tabelle distinte, all'interno dell'entità **Calendario**, potendo usare ID univoco;
- l'entità **Utente** presenta un campo Password criptato tramite funzione di hashing **bcrypt** basata su cifratura Blowfish;
- l'entità **Razza** presenta un campo Lingua che viene utilizzato per popolare l'attributo **lang HTML** e poter quindi rendere accessibile la pronuncia di una razza nel momento in cui viene utilizzata all'interno del sito;
- l'entità **AnimaleRifugio** presenta il campo Età di tipo **FLOAT(2, 2)**, permettendo così di estrapolare dalle cifre unitarie gli anni dell'animali, mentre da quelle decimali i mesi di quest'ultimo; questa scelta permette di filtrare anche per mesi gli animali del Rifugio.

Una copia del DUMP per la base di dati è presente al percorso interno di progetto **Database/dump-rifugio.sql**

4.6 Sicurezza

Per quanto concerne la sicurezza del sito web sono state applicate le seguenti tecniche di prevenzione danni nei confronti delle tecnologie client/server-side:

- creazione, come già parlato precedentemente, della funzione **exeQuery** la quale fa uso di **mysqli_prepare()** per parametrizzare le query e trattare l'input, eccetto dati integer/double, come una semplice stringa; ciò previene SQL Injection;
- password dell'amministratore salvata con tecnica di criptazione tramite funzione di hashing **bcrypt** basata su cifratura Blowfish;
- protezione da attacchi di tipo XSS (Cross-Site Scripting) grazie alla funzione **htmlspecialchars()** inserita all'interno di ogni funzione per l'estrapolazione dei dati dal database ed evitare l'inserimento di elementi HTML che potrebbero rompere la struttura della pagina.

5 Accessibilità e usabilità

Uno degli aspetti fondamentali nella progettazione web risulta essere l'attenzione all'accessibilità, ovvero il ramo che si occupa di rendere utilizzabile un sito web da chiunque. Per questo progetto si è fatto riferimento allo standard WCAG (Web Content Accessibility Guidelines) 2.2, inclusivo dello standard 2.1.

5.1 Elementi fondamentali

L'accessibilità all'interno di questo progetto si concretizza nel rispetto di regole ben specifiche dello standard sopracitato:

- separazione netta tra livello di struttura (HTML5), presentazione (CSS3) e comportamento (JavaScript); questo permette di incorporare elementi di arricchimento delle pagine senza rompere la struttura solida alla base;
- utilizzo di file HTML semantici e validati, per garantire che non si "rompano" alla visualizzazione, rovinando così la lettura tramite tecnologie assistive;
- rispetto della norma WCAG 2.5.8, in quanto tutti gli elementi cliccabili interni al sito presentano un'area cliccabile di almeno 24x24 pixel garantendo il livello minimo AA; i pulsanti, invece, garantiscono un livello di accessibilità AAA con una dimensione minima di 44x44 pixel;
- utilizzo di unità relative all'interno dei fogli di stile per garantire il ridimensionamento degli elementi in base alle necessità dell'utente;
- implementazione di funzionalità "rimuovi sfondo", permettendo ad un utente con difficoltà visive o ADHD di concentrarsi più facilmente nella lettura del testo;

- implementazione di funzionalità "tema chiaro/scuro", permettendo ad utenti ipovedenti (affetti da fotosensibilità, patologie dell'occhio, ...) di navigare il sito con minore difficoltà;

NB Queste ultime due funzionalità vengono rese fruibili anche tramite screen reader, in quanto utili agli utenti ipovedenti per le ragioni soprariportate.

- utilizzo di animazioni con un tempo di esecuzione minimo pari a 0.3s per non inficiare sull'attenzione dell'utente.
- è stato fatto uso di un font accessibile quale **Inter**, privo di grazie per rendere la lettura delle pagine facile a qualsiasi utente e preservare un'estetica pulita;
- validazione dei campi di un form sia a livello client-side che server-side, con particolare attenzione alla stampa degli errori commessi dall'utente, i quali vengono annunciati per essere subito compresi anche da un utente che naviga con tecnologie assistive;
- inserimento di attributi **lang** per l'identificazione della lingua dell'elemento e permettere una lettura efficace da parte dello screen-reader.

5.2 Orientamento e navigazione

Per quanto riguarda questa sezione si elencano le best practices adottate per rendere il sito web di facile navigazione:

- utilizzo dell'elemento breadcrumb, il quale permette all'utente di orientarsi all'interno del sito web;
- implementazione di un pulsante "torna in cima", per facilitare il ritorno alla parte superiore della pagina;
- header e elementi di selezione del contenuto (pannello filtri, pannello di gestione calendario) definiti **sticky** per permetterne la reperibilità durante la navigazione;
- utilizzo di tag **<details>** per la visualizzazione di informazioni principali all'interno di **summary** e dettagli ad esso collegati nella sezione a scomparsa; ciò permette all'utente di navigare con semplicità senza l'obbligo di lettura, soprattutto con screen reader, di un contenuto potenzialmente non richiesto dall'utente stesso;
- elemento di notifica toast, con focus e role **alert** per avvisare l'utente al successo o meno di un'azione interna al sito (es. calendarizzazione di un appuntamento);
- testing ed eventuale correzione della navigazione da tastiera tramite pulsante Tab, con possibile esclusione di alcuni elementi aventi **tabindex=-1**, altrimenti difficili da navigare (es. Google map nella homepage del sito).

5.3 WAI-ARIA

Un'ulteriore pratica adottata all'interno del sito web per renderlo accessibile è l'utilizzo di specifiche WAI-ARIA (Web Accessibility Initiative - Accessible Rich Internet Applications) per definire ruoli e proprietà aggiuntive per rendere la navigazione tramite tecnologie assistive il più comprensibile possibile. Alcuni degli attributi utilizzati sono:

- **aria-label** per la definizione dell'etichetta di pulsanti e renderli così più comprensivi nel loro scopo;
- **aria-hidden** per nascondere elementi, come icone vettoriali, che altrimenti verrebbero percepite dallo screen reader;
- **aria-labelledby** per fornire un titolo significativo a ciascun tag **<section>** all'interno del sito web;
- **aria-expanded** per far comprendere ad un utente ipovedente/non vedente se un elemento si è espanso o meno;
- **aria-live** per avvertire l'utente nel momento in cui il contenuto di un elemento è cambiato dopo una determinata azione;

- attributo **role** per la definizione del ruolo di ciascun elemento e di come debba essere interpretato dalla tecnologia assistiva.

5.4 Colori e Brand Identity

Per quanto concerne l'utilizzo dei colori, quindi di una palette unificata, si è voluto far fronte sia ad un'esigenza estetica che funzionale/accessibile. Nello specifico la palette richiama quelli che sono i colori principali per un'attività a stretto contatto con gli animali, rispettando al tempo stesso quelle che sono le necessità minime di contrasto AA (valore 4.5:1) richieste all'interno dello standard WCAG 2.2. Si riportano di seguito le palette utilizzate con i relativi contrasti.

Tema chiaro	#FFF8E7 (255,248,231)	#F6DFD5 (246,223,213)	#70402B (112,64,43)	#DEE4D2 (222,228,210)	#445D17 (68,93,23)	#EAD6D4 (234,214,212)	#963428 (150,52,40)	#23201F (35,32,31)	#2B7292 (43,114,146)
#FFF8E7 (255,248,231)	-	-	8.07:1	-	7.02:1	-	7.03:1	15.28:1	5.05:1
#F6DFD5 (246,223,213)	-	-	6.68:1	-	5.81:1	-	5.83:1	12.66:1	-
#70402B (112,64,43)	8.07:1	6.68:1	-	6.56:1	-	6.13:1	-	-	-
#DEE4D2 (222,228,210)	-	-	6.56:1	-	5.71:1	-	5.72:1	12.43:1	-
#445D17 (68,93,23)	7.02:1	5.81:1	-	5.71:1	-	5.33:1	-	-	-
#EAD6D4 (234,214,212)	-	-	6.13:1	-	5.33:1	-	5.34:1	11.6:1	-
#963428 (150,52,40)	7.03:1	5.83:1	-	5.72:1	-	5.34:1	-	-	-
#23201F (35,32,31)	15.28:1	12.66:1	-	12.43:1	-	11.6:1	-	-	3.02:1
#2B7292 (43,114,146)	5.05:1	-	-	-	-	-	-	3.02:1	-

Figura 2: Tema chiaro

Tema scuro	#1C1A19 (28,26,25)	#3D2922 (61,41,34)	#C39E8E (195,158,142)	#2A3315 (42,51,21)	#8FA668 (143,166,104)	#3D1C19 (61,28,25)	#D48780 (212,135,128)	#EFEFE6 (239,235,230)	#508FAB (80,143,171)
#1C1A19 (28,26,25)	-	-	7.1:1	-	6.45:1	-	6.25:1	14.61:1	4.83:1
#3D2922 (61,41,34)	-	-	5.59:1	-	5.08:1	-	4.92:1	11.5:1	-
#C39E8E (195,158,142)	7.1:1	5.59:1	-	5.43:1	-	6.23:1	-	-	-
#2A3315 (42,51,21)	-	-	5.43:1	-	4.94:1	-	4.78:1	11.18:1	-
#8FA668 (143,166,104)	6.45:1	5.08:1	-	4.94:1	-	5.67:1	-	-	-
#3D1C19 (61,28,25)	-	-	6.23:1	-	5.67:1	-	5.49:1	12.83:1	-
#D48780 (212,135,128)	6.25:1	4.92:1	-	4.78:1	-	5.49:1	-	-	-
#EFEFE6 (239,235,230)	14.61:1	11.5:1	-	11.18:1	-	12.83:1	-	-	3.02:1
#508FAB (80,143,171)	4.83:1	-	-	-	-	-	-	3.02:1	-

Figura 3: Tema scuro

Com'è possibile notare dalle tabelle soprariportate tutti i contrasti rispettano ciò che è stato descritto nel paragrafo precedente. È stato riportato, all'interno delle tabelle, anche il contrasto tra il colore per i link non visitati (#2B7292 e #508FAB) e il testo normale (#23201F e #EFEFE6), che è il colore dei link già visitati, che è pari a 3.02:1, sufficiente per comprenderne la differenza. Si vuole evidenziare, inoltre, che per l'ideazione del tema scuro non c'è stata una semplice "inversione" dei colori, ma bensì uno studio che garantisse una desaturazione dei colori luminosi, come testo o bordi, in quanto, a differenza del tema

chiaro, i colori chiari hanno una luminosità differente rispetto a quelli scuri, tendendo così a risaltare maggiormente in contrasto con uno sfondo quasi nero. Tutti i colori riportati all'interno di queste palette aiutano a creare così una brand identity per il sito che diventa riconoscibile nella sua grafica e tonalità.

5.5 Convenzioni interne del sito

Per aiutare l'utente durante la navigazione all'interno del sito si è fatto uso delle seguenti convenzioni interne:

- colori di pulsanti, compreso di quest'ultimi sfondo, bordo e testo, sono stati standardizzati con le tonalità del verde, per essere immediatamente identificabili in quanto elementi cliccabili;
- i link non visitati presentano colore azzurro, a differenza di quelli visitati che si "nascondono" riprendendo il colore del testo; entrambi rimangono comunque sottolineati per identificazione immediata da parte dell'utente;
- i link che portano a pagine esterne dal sito vengono segnalati tramite l'uso di un'icona standard nella progettazione web che riporta una freccia che punta verso l'esterno di un quadrato.

6 Test e validazione

Sono stati eseguiti vari test per garantire l'accessibilità del sito e per validare il codice. Di seguito la lista di strumenti che sono stati utilizzati per automatizzare alcuni test:

- **W3C Validator:** strumento che ci ha permesso di analizzare il codice html di ogni pagina per garantire che fosse sotto gli standard di W3C.
- **WAVE Evaluation Tool:** estensione disponibile al browser che ha analizzato la pagina direttamente sotto gli standard alle linee guida di WCAG 2.2.
- **WebAIM Contrast Checker:** strumento utilizzato per verificare il contrasto tra i colori utilizzati nel progetto, garantendo il rispetto del livello di accessibilità AA.

Inoltre, durante la fase dello sviluppo del codice sono stati eseguiti vari test manuali per garantire tutti gli aspetti di accessibilità e funzionalità che non possono essere testati automaticamente:

- **Test con NVDA:** tutte le pagine sono state testate e strutturate correttamente in modo da garantire il giusto funzionamento dello screen reader NVDA. In particolare:
 1. I flag di conteggio nel pannello filtri hanno l'attributo "aria-live=polite" per permette allo screen reader di non fermarsi se il numero si aggiorna dinamicamente mentre sta leggendo la pagina.
 2. Nel pannello filtri per ogni bottone di sezione è stato aggiunto l'attributo "aria-describedby" per informare l'utente di quanti filtri sono stati modificati in quella sezione.
 3. Sono stati inseriti "aria-hidden" per gli elementi che non devono essere letti.
 4. Per le immagini già corredate da figcaption, l'attributo "alt" è stato lasciato vuoto (alt="") per evitare che lo screen reader fornisca una descrizione ridondante o poco rilevante dal punto di vista contestuale.
 5. Ogni termine non italiano è stato marcato con l'attributo "lang".
- **Supporto per tastiera:** si ha prestato attenzione per rendere il sito navigabile da tastiera. Ogni elemento interagibile può essere selezionato senza l'utilizzo di mouse, in particolare:
 1. Ogni elemento dei form ha corretto label associato utilizzando gli attributi "for" nelle label ed "aria-labelledby" nel caso dei gruppi checkbox nel pannello filtri e la barra di ricerca nelle pagine di amministrazione.
 2. I radio button e checkbox sono stati raggruppati correttamente in fieldset con eventuale legenda per descrivere il gruppo.

3. In Ogni campo obbligatorio è stato aggiunto l'attributo "aria-describedby" per avvisare l'utente di eventuali errori nella compilazione dei form in modo dinamico.
 4. Il "tabindex" del pannello filtri e delle relative sezioni è stato modificato per prevenire la navigazione da tastiera quando questi sono chiusi e non visibili.
- **Compatibilità browser multipli:** si ha verificato che il sito fosse correttamente renderizzato ed utilizzabile dai principali browser (Edge, Firefox, Chrome, Brave, Safari),
 - **Compatibilità browser da telefono:** si ha verificato che il sito fosse correttamente renderizzato ed utilizzabile da dispositivi mobili.
 - **Test di reindirizzamento delle pagine:** si ha testato il corretto funzionamento dei reindirizzamenti configurati in .htaccess, sia nella navigazione generale che nelle relative pagine di errore.
 - **Test del layout di stampa:** si ha verificato che durante la stampa la maggior parte dei bottoni venissero nascosti e che non ci fossero elementi tagliati dal cambio di pagina.
 - **Test di funzionalità senza Javascript:** alcuni componenti usufruiscono di codice a lato client per rendere l'esperienza più fluida e animata: in quel caso si ha verificato che il sito funzionasse senza aver bisogno di codice Javascript. In particolare:
 - Ogni campo dei form deve essere verificato sia da lato client con Javascript che da lato server con PHP, rendendo quindi il primo solo un controllo aggiuntivo e non essenziale.
 - Il pannello filtri deve essere nascosto di default, e mostrato solamente se il codice Javascript è attivo, in modo da evitare una esperienza utente incompleta.
 - Il form di porta in adozione deve sostituire il select per la selezione della razza con un input normale, elencando sotto le razze disponibili nel database. La razza inserita sarà effettivamente controllata dal php a lato server.
 - **Test funzionali:** sono stati eseguiti vari test per controllare le seguenti funzionalità:
 1. **Invio e-mail di conferma:** quando l'utente invia un form di adozione o di richiesta porta in adozione, il sistema deve inviare una e-mail di conferma all'indirizzo specificato, con le corrette informazioni e il layout CSS definito dal file style-email.css
 2. **Pagina di amministrazione:** la pagina deve indicare a chi accede quante richieste devono essere gestite per entrambi i tipi (adozione / porta in adozione), e quanti appuntamenti sono programmati per quel giorno.
 3. **Pagine di gestione richieste:** devono essere mostrate nella lista tutte le richieste che non sono state datate nel calendario, e tutte le richieste già assegnate nel calendario che sono previste per una data / ora futura. In particolare:
 - (a) Nella **gestione ticket** (richieste di adozione), devono essere mostrati tutti gli animali disponibili nel centro di adozione. Per ogni animale il sistema deve indicare quante richieste sono da gestire, e quali sono invece già gestite.
 - (b) Nella **gestione richieste inserimento animali in rifugio** invece, devono essere mostrati tutti gli animali "esterni" al rifugio, con aggiuntivo descrittore che indica se la richiesta è stata gestita o no.

Inoltre, ogni richiesta deve essere popolata correttamente dal database (con eventuale immagine dell'animale) e gestita correttamente da tre funzionalità principali:

 - (a) Il pulsante *info* deve aprire un dialog popup che informa di eventuali note della richiesta.
 - (b) Il pulsante *assegnazione a calendario* deve aprire un dialog popup che consenta di selezionare data e ora valide per programmare la visita (con "valide" si intende una data non passata e dentro l'orario lavorativo del centro).
 - (c) Il pulsante *eliminazione* deve aprire un dialog popup che richieda la conferma dell'azione e rimuovi la richiesta dal database.

Se la richiesta è già gestita, i pulsanti (b) e (c) vengono sostituiti da un pulsante che porta al calendario in cui la visita è stata programmata.

4. **Calendario:** la pagina deve essere popolata correttamente dal database, e deve essere possibile navigare tra settimane / mesi. Ogni richiesta deve avere:
 - (a) Il pulsante *info* con funzione identica a quella citata sopra.
 - (b) Il pulsante *modifica appuntamento* che deve aprire un dialog popup per modificare la data e l'ora della visita (ovviamente data e ora valide)
 - (c) Il pulsante *eliminazione* che deve aprire un dialog popup per la conferma della rimozione da calendario (rendendo la richiesta nuovamente da gestire nelle pagine di gestione richieste).
 In caso di data passata l'utente potrà solo verificare le note della prenotazione.
5. **Pannello filtri:** il pannello deve soddisfare le seguenti funzionalità:
 - Se l'accesso avviene tramite desktop il pannello può essere aperto al lato destro della pagina.
 - Se l'accesso avviene tramite dispositivo mobile il pannello può essere aperto come dialog popup.
 - Il pannello deve cambiare il numero di filtri selezionati dinamicamente, sia in maniera generale (dal bottone di "applica filtri"), che per ogni sezione (flag di conteggio).
 - Se il pannello è stato aperto lateralmente deve essere già aperto al caricamento della pagina.
 - I campi vuoti del form del pannello devono essere omessi dalla richiesta GET.
 - I filtri devono essere applicati correttamente nelle pagine che usufruiscono del pannello.
 - Anche se il pannello filtri e la barra di ricerca sono in form separati, la richiesta deve poter tenere eventuali parametri GET già presenti nell'url per evitare la perdita di filtri durante la ricerca e viceversa.
6. **Pagina di adozione:** La pagina deve essere popolata correttamente dal database, e ogni scheda deve portare alla pagina dell'animale selezionato.
7. **Scheda animale:** La pagina deve essere popolata correttamente dal database.
8. **Pagina di porta in adozione:** Il select della razza nel form di richiesta deve popolarsi in modo dinamico in base al tipo di animale scelto tramite richiesta ajax alla pagina razze-json.php
9. **Accesso amministratore:** In caso si navigasse nella pagina di accesso quando si ha già eseguito il login, il sito deve invece mostrare la pagina di logout.
10. **Pagine di amministrazione:** Se l'utente prova ad entrare nelle pagine di amministrazione senza effettuare il login, deve essere reindirizzato alla pagina di errore 401.

7 SEO e prestazioni

Il ranking del sito web, specialmente per quanto riguarda un'attività no-profit e geograficamente circoscritta come il Rifugio in oggetto, è un aspetto fondamentale. Per questo ci si basa sull'incremento della SEO (Search Engine Optimization) per fare in modo che il sito venga proposto ai livelli più alti possibili della SERP (Search Engine Results Page). Le best practices più comuni adottate in questo progetto sono:

- separazione tra struttura (HTML5), presentazione (CSS3) e comportamento (JavaScript) per permettere un caricamento rapido delle pagine del sito.

NB Tramite estensione Chrome Browser Lighthouse è stato possibile verificare una velocità di caricamento della pagina ottimo (99/100), con una velocità di caricamento massima pari a 0.5s.

- uso di formati leggeri per le immagini del sito, come .jpg o WebP;
- inserimento di keywords e metadati dinamici all'interno delle diverse pagine tramite scripting PHP;
- breadcrumb e link di navigazione del sito per permettere agli algoritmi di ispezione delle pagine di analizzare più pagine possibili del sito web;
- uso di tag ed per la marcatura delle keywords all'interno del sito;

- metatag con attributo `property="og:..."` per definire parametri utili nella fase di condivisione del sito tramite socials, come logo, descrizione, ...;
- ottimizzazione del layout responsive per permettere agli algoritmi di ispezione di alzare il livello di ranking;
- tag `<title>` con lunghezza inferiore ai 55 caratteri, dallo specifico al generale e univoco per ciascuna pagina;
- uso di metatag `description` dinamico per ciascuna pagina, il quale non massimizza direttamente il ranking ma potrebbe aiutare nel CTR (Click-Through Rate) aumentando così il numero di interazioni che gli utenti hanno con il sito web;
- dichiarazione degli elementi grafici vettoriali all'interno del file `icons.svg`, la quale scelta permette di diminuire la velocità di caricamento della pagina, permettendo al browser di caricare in un momento unico tutte le icone di cui il sito ha necessità; il vantaggio risiede anche nella manipolazione del colore e nell'organizzazione delle suddette icone vettoriali;
- utilizzo di tag `<h1>` univoco per pagina (logo del sito) contenente la keyword principale, abbinato all'uso gerarchico dei tag `<h2>...<h6>`;
- link ai social media del Rifugio, anche se non concretamente presenti si fa presente che aiutano come elemento per la SEO; concetto analogo è valido anche per l'indirizzo fisico del Rifugio, il quale aiuta alla connessione con ricerche in Google Maps.

Si è deciso inoltre di lavorare sulla SEO semantica, ampliando così il quantitativo di scelte architetture adottate per l'ottimizzazione del ranking. Di seguito viene riportata la descrizione dei file cardine. Nello specifico è stato implementato il vocabolario semantico **Schema.org** attraverso il formato JSON-LD (JavaScript Object Notation for Linked Data), utilizzato per inserire dati strutturati in formato JSON all'interno delle pagine web e renderli facilmente ispezionabili dagli algoritmi di ranking.

7.1 `faq-schema.js`

File "iniettabile" all'interno della homepage, la quale al suo interno presenta FAQs (Frequently Asked Questions) per approfondire eventuali dubbi da parte dell'utente. Lo **Schema.org** in questione si pone l'obiettivo di rendere disponibili ai crawler le domande e le risposte contenute all'interno della pagina, strutturate e leggibili facilmente. Questa sezione aiuta ad aumentare il ranking, in quanto ha la possibilità di abbinare criteri di ricerca generali dell'utente (eseguiti nella barra di ricerca del browser) alla pagina del Rifugio, tramite per l'appunto risposte ad eventuali domande.

7.2 `org-schema.js`

Questo secondo file viene "iniettato" all'interno della homepage, pagina di adozione, pagina di porta in adozione e scheda animale. In questo contesto il file permette di comunicare in modo univoco al browser quelli che sono i dati principali dell'ente, i quali ne identificano l'identità. Alcuni degli attributi definiti nello schema sono: tipologia ente specifica per rifugio animali (**AnimalShelter**), telefono, email, localizzazione geografica, orari di apertura e scopo di lucro dell'ente (\$ ovvero no-profit). Eventualmente è stata aggiunta la possibilità di collegamento ai link social anche se non presenti nel concreto. Questi dati aiutano a creare autorevolezza per l'ente del Rifugio, permettendo così una massimizzazione del ranking all'interno della SERP.

8 Organizzazione del lavoro

Ogni membro ha contribuito alla progettazione del database e alla scelta dei colori utilizzati nel sito, valutati in base al contrasto secondo gli standard delle linee guida WCAG 2.2, con Thomas Morettin come principale contributore in questo ambito. Per quanto riguarda lo sviluppo delle pagine, il gruppo si è organizzato per la realizzazione del progetto suddividendo il lavoro in base alle pagine da realizzare, ognuna delle quali prevedeva lo sviluppo di HTML, CSS, PHP e JavaScript:

- Thomas Morettin: header e footer, pagina *Index*, pagina *Calendario*, pagina *Gestione ticket*, pagina *Amministrazione*.

- Felician Mario Necsulescu: pagina *Adotta*, pagina *Scheda animale*
- Niccolò Feltrin: pagina *Accesso*, pagina *Porta in adozione*, pagina *Gestione richieste inserimento rifugio*

A livello funzionale, sono state assegnate:

- a Thomas Morettin il sistema di invio e-mail.
- a Niccolò Feltrin il pannello filtri utilizzabile modularmente.
- a Felician Mario Necsulescu la maggior parte della validazione dei campi nei form presente nel sito.

Si ha data quindi la responsabilità ad ogni membro del gruppo di validare l'accessibilità delle proprie pagine e di testarne il corretto funzionamento.

9 Conclusioni e difficoltà incontrate

In conclusione, il sito è stato progettato in un contesto il più possibile realistico, con l'obiettivo di sviluppare un sistema in grado di gestire in modo completo tutte le transazioni (adozioni e appuntamenti) di un centro di adozione immaginario. Particolare attenzione è stata posta affinché il sistema risultasse accattivante, accessibile, semplice da utilizzare, immediato e distintivo. Il progetto ci ha permesso di esplorare diverse fasi dello sviluppo web, dalla progettazione del logo alla definizione delle funzionalità principali del sito, offrendo un'esperienza completa e pratica che ha arricchito le competenze di tutti i membri, sia sul design pulito e funzionale, sia sulla realizzazione di un servizio web operativo. L'attività ha inoltre fornito l'opportunità di gestire efficacemente il lavoro di gruppo, sperimentando la suddivisione dei compiti e l'organizzazione delle pagine. Particolarmente significativa è stata la progettazione di un sistema modulare per la creazione delle pagine e l'implementazione di un meccanismo di invio automatico delle e-mail, che ha consentito di rendere il sito più completo e professionale. Grazie alla sua struttura modulare, il progetto può essere facilmente esteso e adattato per un'implementazione reale in un centro di adozione, offrendo così un'esperienza arricchente per i membri del gruppo.

Alcune difficoltà sono state riscontrate durante la progettazione del sito:

- **Accessibilità:** in alcune funzionalità del sito, come nelle pagine di gestione delle richieste, nelle schede animali o nel pannello dei filtri, è stato particolarmente impegnativo cercare di trovare un bilanciamento tra estetica accattivante e layout accessibile, portandoci a studiare esaustivamente diverse opzioni di progettazione e prendendo decisioni in fase d'implementazione che scartavano delle scelte prese all'inizio del progetto per un approccio più accessibile o modulare.
- **Unificazione del sito:** vista l'organizzazione dello sviluppo delle pagine, nettamente suddivisa tra i membri, è stato dedicato del tempo per rendere l'intero sito un'esperienza coerente e uniforme, un aspetto che non era immediato all'inizio della progettazione. Per questo il gruppo ha pianificato vari incontri che permettessero di dialogare su proposte e/o consigli in modo da accordarsi sulle decisioni più importanti.
- **Previsione del carico di lavoro:** anche se fin dall'inizio l'obiettivo del sito era chiaro al gruppo, è capitato più volte di proporre idee troppo complesse da implementare o troppo dispendiose in termini di tempo. È stato quindi necessario rivalutare alcune scelte originali per garantire la fattibilità del progetto.